

Multi-Bank Statement Parser — Complete Implementation Guide

Scope: Extend the existing single/two-bank Excel + PDF statement import feature into a scalable, multi-bank system that auto-detects file type, auto-detects the issuing bank, routes to the correct parser, normalizes the output, cleans narrations, and feeds the existing HTML table + voucher-creation workflow.

Stack assumed: Nuxt 3/4 (Nitro server routes), MongoDB/Mongoose, `exceljs` (already installed), `pdfjs-dist` (already installed), existing narration-cleaning logic, existing dynamic table + voucher modal UI.

0. Design Principles (read this first)

1. One normalized output schema, many input adapters.

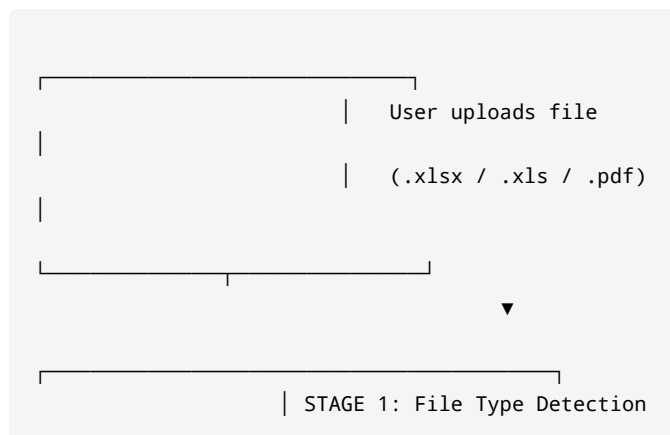
Every bank/format ultimately produces the same shape of data. Nothing downstream of parsing should ever know or care which bank the file came from.

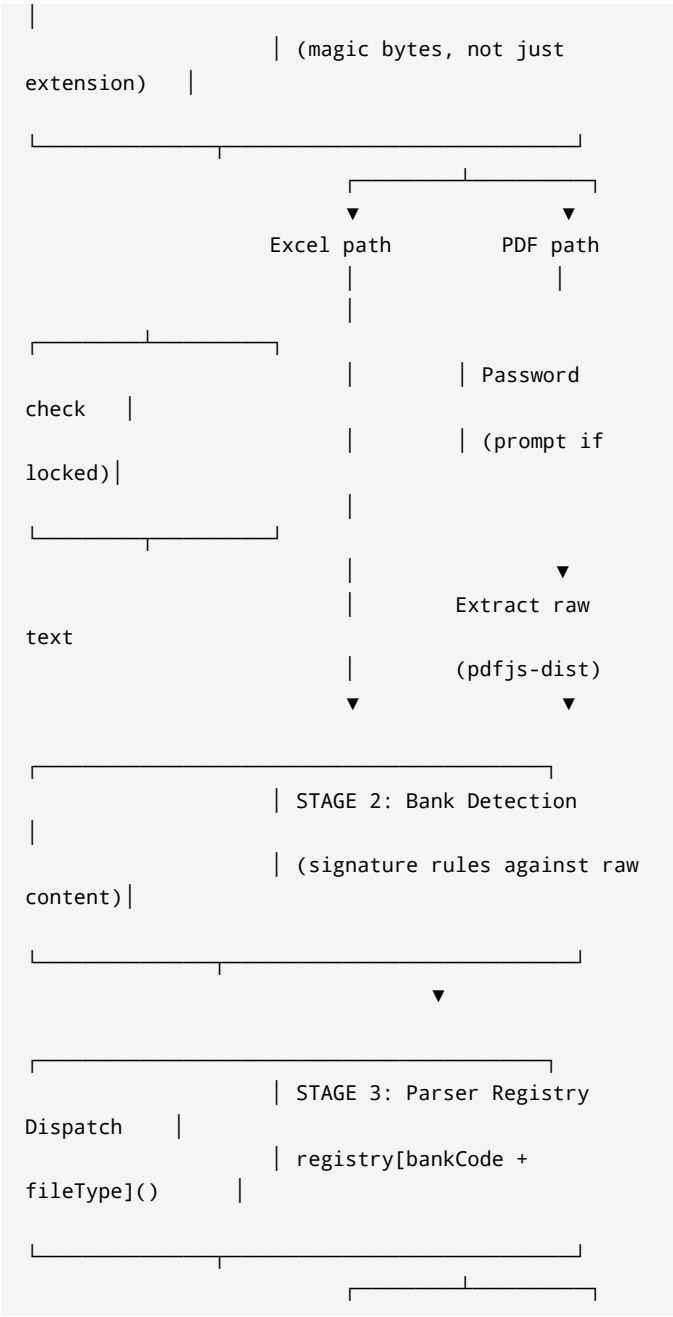
2. Detection is a pipeline, not a single guess.

File type detection and bank detection are two separate, sequential steps — each with its own fallback behavior.

3. **Never trust silently.** If detection is uncertain, or a bank has no dedicated parser, fall back to a best-effort generic parser and flag the result as "unverified — please review" rather than either failing hard or pretending it's accurate.
4. **Grow the parser library driven by real usage.** Don't try to pre-build parsers for every Indian bank on day one. Build for the banks your users actually upload, add new adapters as new formats show up in production.
5. **This mirrors a pattern you already use.** Your multi-provider backup system (Infini-Cloud / Vercel Blob / B2 behind a common interface with isolated failure per provider) is architecturally the same shape as this: shared orchestration, swappable provider-specific adapters, isolated failure per adapter.

1. High-Level Architecture






```
(existing UI)      |
                   | → Voucher Creation Modal
(existing)        |
└──────────────────┘
```

2. Stage 1 — File Type Detection

Don't trust the file extension alone (users rename files, browsers sometimes report wrong MIME types). Detect by **magic bytes** first, extension as a secondary hint.

Format	Magic bytes (hex)	Notes
<code>.xlsx</code>	50 4B 03 04 (PK..)	It's a ZIP container; exceljs handles it directly
<code>.xls</code> (legacy)	D0 CF 11 E0 A1 B1 1A E1	OLE2 format — exceljs does not read this natively; see note below
<code>.pdf</code>	25 50 44 46 (%PDF)	pdfjs-dist handles it directly

```
// server/utils/statementParser/detectFileType.ts
export type DetectedFileType = 'xlsx' | 'xls-legacy' | 'pdf' | 'unknown'

export function detectFileType(buffer: Buffer): DetectedFileType {
  const hex = buffer.subarray(0,
```

```

8).toString('hex').toUpperCase()
  if (hex.startsWith('504B0304')) return 'xlsx'
  if (hex.startsWith('D0CF11E0A1B11AE1')) return
'xls-legacy'
  if (hex.startsWith('25504446')) return 'pdf'
  return 'unknown'
}

```

Important gotcha: some Indian bank "Excel" exports are actually **HTML tables saved**

with an .xls extension, not real binary Excel files (SBI is notorious for this on

older net-banking exports). Add a check: if magic bytes don't match either Excel format but

the extension is .xls / .xlsx, sniff the first ~200 bytes as text — if it starts with

<html, <table, or <?xml, route it to an **HTML-table parser** instead of exceljs.

```

function isHtmlDisguisedAsExcel(buffer: Buffer):
boolean {
  const head = buffer.subarray(0,
300).toString('utf-8').trim().toLowerCase()
  return head.startsWith('<html') ||
head.startsWith('<table') || head.startsWith('<?
xml')
}

```

3. Stage 1b — Password-Protected PDF Handling

Most Indian bank statement PDFs downloaded via net banking/app are **password protected**

(typically DOB in `DDMMYYYY` format, or Customer ID for HDFC, or account number for SBI NetBanking). Your pipeline must handle this explicitly rather than let parsing fail silently.

```
import * as pdfjsLib from 'pdfjs-  
dist/legacy/build/pdf.mjs'  
  
async function tryLoadPdf(buffer: Uint8Array,  
password?: string) {  
  try {  
    const doc = await pdfjsLib.getDocument({ data:  
buffer, password }).promise  
    return { doc, requiresPassword: false }  
  } catch (err: any) {  
    if (err.name === 'PasswordException') {  
      return { doc: null, requiresPassword: true }  
    }  
    throw err  
  }  
}
```

UX flow:

1. User uploads PDF.
2. Server attempts to open it with no password.
3. If `PasswordException` is thrown, respond with a distinct status (e.g. `423 Locked` or a custom `{ requiresPassword: true }` payload) — **do not treat this as a generic parse failure.**
4. Client shows a "This PDF is password protected — enter the password" prompt.
5. Re-submit with password; retry `getDocument({ data, password })`.

6. Never log or store the password. Don't persist the unlocked PDF bytes longer than the request lifecycle unless the user explicitly wants the file archived (tie into your existing Documents/Backblaze module if so – store the **unlocked** version there so the user never re-enters the password for the same file).
-

4. Stage 2 – Bank Detection

Detection works differently for Excel vs. PDF, but both funnel into the same output: a

```
bankCode string (e.g. 'SBI' , 'HDFC' , 'ICICI' , 'AXIS' ,  
'BOI' , 'PNB' , 'BOB' ,  
'KOTAK' , 'UNKNOWN' ).
```

4.1 Detection signals (in priority order)

1. **Explicit bank name/logo text** near the top of the document – most reliable.

PDFs: check the first page's extracted text for known strings ("State Bank of India" ,

```
"HDFC BANK" , "ICICI Bank Limited" , "AXIS BANK" ,  
"Bank of India" , "Punjab National Bank" , "Bank of  
Baroda" , "KOTAK MAHINDRA BANK" ).
```

Excel: check cell values in the first ~10 rows / first sheet's header/footer.

2. **IFSC code prefix** – every Indian bank's IFSC starts with a unique 4-letter bank

```
code ( SBIN , HDFC , ICIC , UTIB for Axis, BKID for BOI,  
PUNB , BARB , KKBK ).
```

If the statement includes IFSC (many do, in the header),

this is a very reliable signal
since it's a structured code, not free text prone to
OCR/formatting noise.

3. **Column header signature** — each bank's table headers
differ slightly even when the
underlying data is conceptually the same:

- SBI: Txn Date | Value Date | Description | Ref
No./Cheque No. | Debit | Credit | Balance
 - HDFC: Date | Narration | Chq./Ref.No. | Value
Dt | Withdrawal Amt. | Deposit Amt. | Closing
Balance
 - ICICI: Value Date | Transaction Date | Cheque
Number | Transaction Remarks | Withdrawal
Amount (INR) | Deposit Amount (INR) | Balance
(INR)
 - Axis: Tran Date | Chq No | Particulars | Debit
| Credit | Balance | Init. Br
- Store these as a small signature table and match
against extracted header text.

4. **Narration format micro-patterns** (last-resort, lowest
confidence) — e.g. HDFC UTR
narrations use forward slashes, ICICI uses hyphens, Axis
uses plain spaces around
reference numbers. Only use this as a tie-breaker if signals
1–3 are ambiguous.

```
// server/utils/statementParser/detectBank.ts
interface BankSignature {
  code: string
  nameMatches: RegExp[]
  ifscPrefix: string
  headerSignature: RegExp // matches a
```

```

serialized/joined header row
}

const BANK_SIGNATURES: BankSignature[] = [
  {
    code: 'SBI',
    nameMatches: [/state bank of india/i,
/\bSBI\b/],
    ifscPrefix: 'SBIN',
    headerSignature: /txn date.*value
date.*description/i,
  },
  {
    code: 'HDFC',
    nameMatches: [/hdfc bank/i],
    ifscPrefix: 'HDFC',
    headerSignature:
/narration.*chq.*ref.*withdrawal amt/i,
  },
  {
    code: 'ICICI',
    nameMatches: [/icici bank/i],
    ifscPrefix: 'ICIC',
    headerSignature: /transaction
remarks.*withdrawal amount.*deposit amount/i,
  },
  {
    code: 'AXIS',
    nameMatches: [/axis bank/i],
    ifscPrefix: 'UTIB',
    headerSignature: /tran
date.*particulars.*debit.*credit/i,
  },
  {
    code: 'BOI',
    nameMatches: [/bank of india/i],
    ifscPrefix: 'BKID',
    headerSignature: /.*/,    // fill in once you
have a real BOI sample
  },

```

```

    // add PNB, BOB, KOTAK, etc. as you encounter
    real samples
  ]

  export function detectBank(rawText: string): string
  {
    const lower = rawText.toLowerCase()

    // Priority 1+2: name text and IFSC prefix
    for (const sig of BANK_SIGNATURES) {
      if (sig.nameMatches.some(re =>
re.test(rawText))) return sig.code
      const ifscMatch = rawText.match(/IFSC\s*[:\-\]?
\s*([A-Z]{4})/i)
      if (ifscMatch && ifscMatch[1].toUpperCase() ===
sig.ifscPrefix) return sig.code
    }

    // Priority 3: header signature
    for (const sig of BANK_SIGNATURES) {
      if (sig.headerSignature.test(lower)) return
sig.code
    }

    return 'UNKNOWN'
  }

```

Confidence scoring (recommended refinement): rather than returning the first match, score each signal (name match = high confidence, header-only match = medium, no match = low) and surface the confidence level to the UI. If confidence is "medium" or lower, show a "We think this is a [Bank] statement — confirm?" dropdown before running the bank-specific parser, defaulting to the detected bank but letting the user override. This single UI

safeguard prevents a wrong auto-detect from silently mis-mapping columns.

5. Stage 3 – Parser Registry & Dispatch

One registry, keyed by `bankCode` + `fileType`, mapping to a parser function. This is the central "smart feature" that calls the appropriate script.

```
// server/utils/statementParser/registry.ts
import { parseSbiExcel } from './parsers/sbi.excel'
import { parseSbiPdf } from './parsers/sbi.pdf'
import { parseBoiExcel } from './parsers/boi.excel'
import { parseBoiPdf } from './parsers/boi.pdf'
import { parseHdfcPdf } from './parsers/hdfc.pdf'
import { parseGenericExcel } from
'./parsers/generic.excel'
import { parseGenericPdf } from
'./parsers/generic.pdf'

export type RawTransaction = {
  date: string
  narration: string
  refNo?: string
  debit?: number
  credit?: number
  balance?: number
}

export type ParserFn = (input: ParserInput) =>
Promise<RawTransaction[]>

interface ParserInput {
  buffer: Buffer
  rawText?: string    // for PDFs, pre-extracted
  text
```

```

    workbookMeta?: any      // for Excel, sheet/cell
    context if needed
  }

  type RegistryKey = `${string}:${'xlsx' | 'pdf'}`

  const registry = new Map<RegistryKey, ParserFn>([
    ['SBI:xlsx', parseSbiExcel],
    ['SBI:pdf', parseSbiPdf],
    ['BOI:xlsx', parseBoiExcel],
    ['BOI:pdf', parseBoiPdf],
    ['HDFC:pdf', parseHdfcPdf],
    // add more as adapters are built
  ])

  export function getParser(bankCode: string,
    fileType: 'xlsx' | 'pdf'): {
    parser: ParserFn
    isFallback: boolean
  } {
    const key = `${bankCode}:${fileType}` as
    RegistryKey
    const found = registry.get(key)
    if (found) return { parser: found, isFallback:
    false }

    const fallback = fileType === 'xlsx' ?
    parseGenericExcel : parseGenericPdf
    return { parser: fallback, isFallback: true }
  }

```

This design means **adding a new bank never touches shared code** — you write one new file (`newbank.pdf.ts` or `newbank.excel.ts`), register it, done. Nothing else in the pipeline changes.

6. Stage 3b — Individual Parser

Implementation Pattern

Every parser, regardless of bank, follows the same internal shape: locate the transaction table, iterate rows, map bank-specific columns to the common shape. Only the column mapping/regex differs.

6.1 Excel parser example (exceljs)

```
//
server/utils/statementParser/parsers/sbi.excel.ts
import ExcelJS from 'exceljs'
import type { RawTransaction } from '../registry'

export async function parseSbiExcel({ buffer }: {
  buffer: Buffer }): Promise<RawTransaction[]> {
  const wb = new ExcelJS.Workbook()
  await wb.xlsx.load(buffer)
  const sheet = wb.worksheets[0]

  const transactions: RawTransaction[] = []
  let headerRowIndex = -1

  // Locate header row dynamically – SBI exports
  // often have several
  // metadata rows above the actual table (account
  // no, branch, period, etc.)
  sheet.eachRow((row, rowNumber) => {
    const rowText =
      row.values?.toString().toLowerCase() ?? ''
    if (rowText.includes('txn date') &&
      rowText.includes('description')) {
      headerRowIndex = rowNumber
    }
  })
}
```

```

    if (headerRowIndex === -1) throw new Error('SBI
header row not found – format may have changed')

    for (let i = headerRowIndex + 1; i <=
sheet.rowCount; i++) {
      const row = sheet.getRow(i)
      const date = row.getCell(1).text?.trim()
      if (!date) continue // skip blank/footer rows

      transactions.push({
        date,
        narration: row.getCell(3).text?.trim() ?? '',
        refNo: row.getCell(4).text?.trim(),
        debit: parseAmount(row.getCell(5).text),
        credit: parseAmount(row.getCell(6).text),
        balance: parseAmount(row.getCell(7).text),
      })
    }

    return transactions
  }

function parseAmount(val?: string): number |
undefined {
  if (!val) return undefined
  const cleaned = val.replace(/,/g, '').trim()
  const num = parseFloat(cleaned)
  return isNaN(num) ? undefined : num
}

```

6.2 PDF parser example (pdfjs-dist)

PDF text extraction gives you a flat stream of text items with x/y coordinates — you need to reconstruct rows by grouping items that share a similar y-coordinate, then sort by x-coordinate within each row.

```
//
server/utils/statementParser/parsers/pdfTableExtractor.ts
import * as pdfjsLib from 'pdfjs-
dist/legacy/build/pdf.mjs'

interface TextItemPos { text: string; x: number; y:
number }

export async function extractPdfRows(buffer:
Uint8Array, password?: string):
Promise<TextItemPos[][]> {
  const doc = await pdfjsLib.getDocument({ data:
buffer, password }).promise
  const allRows: TextItemPos[][] = []

  for (let p = 1; p <= doc.numPages; p++) {
    const page = await doc.getPage(p)
    const content = await page.getTextContent()

    const items: TextItemPos[] =
content.items.map((it: any) => ({
      text: it.str,
      x: it.transform[4],
      y: it.transform[5],
    })))

    // group by y-coordinate (rows), tolerant of
small pixel drift
    const rowsMap = new Map<number, TextItemPos[]>
()
    for (const item of items) {
      const yKey = Math.round(item.y / 3) * 3 //
bucket tolerance
      if (!rowsMap.has(yKey)) rowsMap.set(yKey, [])
      rowsMap.get(yKey)!.push(item)
    }

    const sortedRows = [...rowsMap.entries()]
  }
}
```



```

        .sort((a, b) => b[0] - a[0])    // top to
bottom (PDF y is bottom-up)
        .map(([, rowItems]) => rowItems.sort((a, b)
=> a.x - b.x))

        allRows.push(...sortedRows)
    }

    return allRows
}

```

```

// server/utils/statementParser/parsers/sbi.pdf.ts
import { extractPdfRows } from
'./pdfTableExtractor'
import type { RawTransaction } from '../registry'

const DATE_RE = /^\\d{1,2}[-/][A-Za-z]{3}
[-/]\\d{2,4}$|^\\d{1,2}[-/]\\d{1,2}[-/]\\d{2,4}$/

export async function parseSbiPdf({ buffer,
password }: any): Promise<RawTransaction[]> {
    const rows = await extractPdfRows(buffer,
password)
    const transactions: RawTransaction[] = []

    for (const row of rows) {
        const joined = row.map(r => r.text).join('
').trim()
        const firstToken = row[0]?.text?.trim()

        if (!firstToken || !DATE_RE.test(firstToken))
continue // not a transaction row

        // SBI PDF column order after date: description
... ref ... debit credit balance
        // Amounts are right-aligned – identify them by
numeric pattern from the end
        const amounts = extractTrailingAmounts(row, 3)
// expect up to 3 numeric columns

```

```

    const narration = row.slice(1, row.length -
amounts.length).map(r => r.text).join(' ').trim()

    transactions.push({
      date: firstToken,
      narration,
      debit: amounts[0],
      credit: amounts[1],
      balance: amounts[2],
    })
  }

  return transactions
}

function extractTrailingAmounts(row: { text: string
}[], maxCount: number): (number | undefined)[] {
  const numeric = /^[\\d,]+\\.\\d{2}$/
  const trailing = [...row].reverse().filter(r =>
numeric.test(r.text.trim())).slice(0, maxCount)
  return trailing.reverse().map(r =>
parseFloat(r.text.replace(/,/g, ' ')))
}

```

Reality check: PDF table reconstruction is inherently the messiest part of this whole system. Expect to spend more iteration time here than anywhere else — every bank (and sometimes different account types within the same bank, e.g. savings vs. current vs. NRI) can shift column widths, wrap narrations onto a second line, or omit a column when it's zero. Build this parser against 3–5 **real, sample statements per bank** (redact account numbers before using them in test fixtures) rather than guessing from documentation alone.

6.3 Generic fallback parser

For any bank without a dedicated adapter, attempt a best-effort extraction assuming the common Indian-bank pattern (date, narration, debit, credit, balance — 5-ish columns) and flag the result:

```
//
server/utils/statementParser/parsers/generic.pdf.ts
export async function parseGenericPdf(input: any):
Promise<RawTransaction[]> {
  // same row-grouping approach as sbi.pdf.ts, but
  with looser column assumptions
  // and no bank-specific narration parsing
  // ...
  // Every transaction from this parser should be
  tagged for the UI:
  // __unverified: true
  return transactions.map(t => ({ ...t,
    __unverified: true }))
}
```

In the UI, rows from the fallback parser should render with a visible "unverified — please check amounts/dates" badge, and ideally the raw extracted text should be viewable side-by-side so the user can manually correct misaligned columns before posting to accounting.

7. Stage 4 — Normalization

Whatever the parser returns, immediately convert it into one strict, final schema before it touches narration cleaning or the UI. This is the seam that makes stages 5–7 bank-agnostic.

```
// server/utils/statementParser/normalize.ts
export interface NormalizedTransaction {
  date: Date
  rawNarration: string
  cleanedNarration?: string // filled in by
Stage 5
  refNo?: string
  debit: number // 0 if not
applicable
  credit: number // 0 if not
applicable
  balance?: number
  bankCode: string
  sourceFileType: 'xlsx' | 'pdf'
  unverified: boolean
  rowHash: string // for de-dup,
see Stage 6
}

export function normalize(
  raw: RawTransaction[],
  bankCode: string,
  sourceFileType: 'xlsx' | 'pdf',
  unverified: boolean
): NormalizedTransaction[] {
  return raw.map(r => {
    const date = parseIndianDate(r.date)
    const debit = r.debit ?? 0
    const credit = r.credit ?? 0
    return {
      date,
      rawNarration: r.narration,
      refNo: r.refNo,
      debit,
```

```

        credit,
        balance: r.balance,
        bankCode,
        sourceFileType,
        unverified,
        rowHash: computeRowHash(date, r.narration,
debit, credit),
    }
    })
}

function parseIndianDate(input: string): Date {
    // Handle common Indian bank date formats:
    // DD/MM/YYYY, DD-MM-YYYY, DD-MMM-YYYY, DD MMM
YYYY
    // Implement explicitly rather than relying on
`new Date(str)`,
    // which misparses DD/MM as MM/DD in some
environments.
    // ... (use a small parsing utility or dayjs with
explicit format strings)
}

function computeRowHash(date: Date, narration:
string, debit: number, credit: number): string {
    const crypto = require('node:crypto')
    return crypto.createHash('sha256')

.update(`${date.toISOString()}|${narration}|${debit
}|${credit}`)
    .digest('hex')
}

```

Why `rowHash` matters: if the same statement (or an overlapping date range from a second export) gets uploaded twice, this hash lets Stage 6 detect and warn about duplicates

before the user accidentally posts the same transaction to accounting twice.

8. Stage 5 — Narration Cleaning (reuse existing logic)

You already have this built and working for 2 banks — the key requirement for this redesign is to make it **bank-agnostic and composable**, not bank-specific:

```
// server/utils/statementParser/cleanNarration.ts
export function cleanNarration(rawNarration:
string, bankCode: string): string {
  let text = rawNarration

  // 1. Shared, universal cleanup (applies to every
bank)
  text = stripCommonNoise(text) // your existing
generic junk-stripping logic

  // 2. Bank-specific narration quirks (optional,
only if needed)
  const bankSpecificCleaner =
bankNarrationCleaners[bankCode]
  if (bankSpecificCleaner) text =
bankSpecificCleaner(text)

  return text.trim()
}

const bankNarrationCleaners: Record<string, (t:
string) => string> = {
  HDFC: (t) => t.replace(/\\INF\\[A-Z0-9]+/g, ''),
  // HDFC-specific reference blocks
```

```
ICICI: (t) => t.replace(/-TXT-[A-Z0-9]+/g, ''),
// ICICI-specific reference blocks
// add as you observe real bank-specific junk
patterns
}
```

Since real Indian bank narrations follow the pattern `RAIL/UTR-` or `VPA/COUNTERPARTY/EXTRA` (UPI, NEFT, IMPS, RTGS as described in section 4), a strong universal cleaning strategy is:

- strip known rail keywords (`UPI` , `NEFT` , `IMPS` , `RTGS` , `NACH`),
- strip long numeric/
- alphanumeric reference blocks (UTRs are always 22 chars, UPI transaction IDs are 12 digits, IMPS RRNs are 12 digits), strip VPA suffixes (`@okaxis` , `@ybl` , `@paytm` , etc. — but keep
- the handle prefix, since that's often the recognizable name), and
- keep whatever plain-text
- counterparty name remains.

9. Stage 6 — Post-Processing

9.1 Duplicate detection

Compare `rowHash` against previously imported transactions for the same firm (query your accounting/bank-import collection). Flag matches as "possible duplicate" in the UI rather than silently blocking — the user might legitimately be re-importing an overlapping date range on purpose.

9.2 Running balance validation

If the statement includes a balance column, validate that `previousBalance + credit - debit == currentBalance` for consecutive rows. Mismatches usually mean a parsing error (columns misaligned, a row was skipped, or a multi-line narration ate part of the next row) — surface this as a per-row warning icon rather than failing the whole import. This is one of the most useful integrity checks you can build, since it catches parser bugs automatically on every real-world file, not just your test fixtures.

9.3 Party auto-matching

This is a natural connection point to your GSTIN/party lookup work — once the narration is cleaned down to a counterparty name, do a fuzzy match (e.g. `string-similarity` or simple normalized substring match) against your existing `Party` collection for that firm. If a strong match is found, pre-select that party in the voucher creation modal instead of leaving it blank — meaningfully speeds up the one-by-one posting step you already have.

10. Stage 7 — UI Integration (extends what you have)

Your existing HTML table + voucher modal stays the same conceptually — extend it with a few new signals surfaced from the pipeline above:

- **Bank badge** per import batch (e.g. "Detected: HDFC Bank – 94% confidence")
- **Unverified badge** on rows from the generic fallback parser
- **Duplicate warning** icon per row (from 9.1)
- **Balance mismatch warning** icon per row (from 9.2)
- **Suggested party** pre-filled in the voucher modal (from 9.3), still editable
- **Manual bank override dropdown** if detection confidence is medium/low (from Section 4)
- **"Re-upload with password" prompt** for locked PDFs (from Section 3)

11. Suggested API Route Shape

```
// server/api/accounting/bank-import/parse.post.ts
export default defineEventHandler(async (event) => {
  {
    const form = await readMultipartFormData(event)
    const file = form?.find(f => f.name ===
'statement')
    const password = form?.find(f => f.name ===
'password')?.data?.toString()

    if (!file) throw createError({ statusCode: 400,
message: 'No file uploaded' })

    const fileType = detectFileType(file.data)

    if (fileType === 'pdf') {
      const { requiresPassword } = await
tryLoadPdf(file.data, password)
      if (requiresPassword && !password) {
        return { requiresPassword: true }
      }
    }
  }
})
```

```

    }
  }

  const rawText = fileType === 'pdf'
    ? await extractRawTextForDetection(file.data,
password)
    : await extractExcelTextForDetection(file.data)

  const { bankCode, confidence } =
detectBank(rawText)
  const parserFileType = fileType === 'xls-legacy'
? 'xlsx' : fileType as 'xlsx' | 'pdf'
  const { parser, isFallback } =
getParser(bankCode, parserFileType)

  const raw = await parser({ buffer: file.data,
password })
  const normalized = normalize(raw, bankCode,
parserFileType, isFallback)
  const cleaned = normalized.map(t => ({
    ...t,
    cleanedNarration:
cleanNarration(t.rawNarration, bankCode),
  }))

  const withChecks = await
runPostProcessing(cleaned, event.context.firmId)

  return {
    bankCode,
    confidence,
    isFallbackParser: isFallback,
    transactions: withChecks,
  }
})

```

12. Testing Strategy

- **Fixture-based unit tests per parser.** Store 3–5 redacted real sample statements per supported bank under `test/fixtures/statements/{bank}/`. Redact account numbers, names, and amounts if these are real personal files — synthetic-but-structurally-accurate fixtures work too if you don't want to store real data even redacted.
 - **Golden-output comparison.** For each fixture, hand-verify the expected parsed output once, store it as a JSON snapshot, and assert new parser changes don't silently break previously-working extraction (classic snapshot/regression testing).
 - **Detection accuracy test.** A separate test suite that only exercises `detectFileType` and `detectBank` against all fixtures, asserting correct bank identification independent of whether the full parse succeeds.
 - **Fallback path test.** At least one fixture that's deliberately from an unsupported bank, asserting it correctly routes to the generic parser and gets flagged `unverified: true`.
-

13. Rollout Plan

1. Build the shared pipeline skeleton (Stages 1, 2 detection only — no real parsers yet) using your two existing banks (SBI, BOI) as the first two

registry entries, migrating

your current logic into the `sbi.excel.ts` / `boi.excel.ts`
/ `boi.pdf.ts` adapter shape.

2. Verify parity — the new pipeline should produce identical output to your current feature for these two banks before adding anything new.
3. Add the generic fallback parser + "unverified" UI badge — this alone immediately gives you support for "any other bank," even before writing a dedicated adapter, just with lower accuracy.
4. Add password-protected PDF handling (Section 3) — likely to affect a large share of real user uploads regardless of which bank.
5. Add duplicate detection + running balance validation (Section 9) — highest value-per-effort post-processing checks.
6. Prioritize dedicated adapters for the next 2–3 most-requested banks based on actual user uploads hitting the fallback parser — this is your real-world signal for where to invest next, rather than guessing.
7. Add party auto-matching (ties into your GSTIN/party work) once the core pipeline is stable.

14. Summary of Additional Features

Beyond the Original Request

These weren't explicitly asked for but are natural, high-value extensions given what this

pipeline touches:

- Password-protected PDF handling with a proper unlock UX (near-certain real-world need)
- Detection confidence scoring + manual override (prevents silent mis-parsing)
- Generic fallback parser with "unverified" flagging (immediate any-bank support)
- Duplicate transaction detection via row hashing
- Running balance validation as an automatic integrity check
- Party auto-matching from cleaned narration into your existing Party/GSTIN system
- Fixture-based regression testing per bank (protects against silent breakage as banks change their statement templates over time — this *will* happen periodically)
- Optional: archive the unlocked PDF into your existing Documents/Backblaze module so users never have to re-enter a statement password for the same file

End of guide.